ByteMart Digital

THE COMPLETE GUIDE

Java 25

Modern Java: The Complete Guide to the Newest LTS

A complete, example-driven guide to modern Java through the Java 25 LTS — virtual threads, scoped values, pattern matching, records, stream gatherers, runtime performance, and migration.

24	120+	12	6
Chapters	Code Examples	Diagrams	Modules

© 2026 ByteMart Digital · All Rights Reserved

ByteMart Digital

Java 25: The Complete Guide

First Edition · 2026

Copyright © 2026 ByteMart Digital. All rights reserved.
No part of this publication may be reproduced, distributed, or transmitted in any form or by any means — including photocopying, recording, screenshotting, or other electronic or mechanical methods — without the prior written permission of the publisher, except for brief quotations used in reviews and certain other non-commercial uses permitted by copyright law.

This product is licensed for personal use by the purchaser only. Sharing, reselling, or redistributing this file in whole or in part is strictly prohibited and constitutes a violation of copyright law.

Disclaimer
The information in this book is provided in good faith and for educational purposes. While every effort has been made to ensure accuracy, the author and publisher assume no responsibility for errors, omissions, or outcomes arising from the use of the code or information contained herein. All code examples are provided "as is".

Trademarks
Java, OpenJDK, and other product names, logos, and brands referenced are the property of their respective owners and are used for identification purposes only. This guide is an independent educational work.

Published by ByteMart Digital

Web: bytemartdigital.com
Email: contact@bytemartdigital.in
Instagram: instagram.com/bytemartdigital.in
Telegram: t.me/bytemartcodingcommunity

Contents

1. Java 25 & the LTS Model 4
2. The Journey from 21 to 25 6
3. Simpler First Programs 8
4. Module Imports & Flexible Constructors 11
5. Virtual Threads: The Big Idea 14
6. Using Virtual Threads 16
7. Virtual Threads in Practice 18
8. Structured Concurrency 20
9. Scoped Values 23
10. Record Patterns 26
11. Pattern Matching for switch 29
12. Primitive Types in Patterns 32
13. Sealed + Records + Patterns 35
14. Sequenced Collections 37
15. Stream Gatherers 40
16. The Modern Java Basics 43
17. Stable Values 45
18. Foreign Function & Memory API 48
19. Compact Object Headers 50
20. Modern Garbage Collectors 52
21. Faster Startup: AOT & Leyden 54
22. Observability: JFR & Class-File API 57
23. Migrating to Java 25 59
24. Best Practices & Modern Java Style 61

MODULE 1 · THE JAVA 25 LANDSCAPE

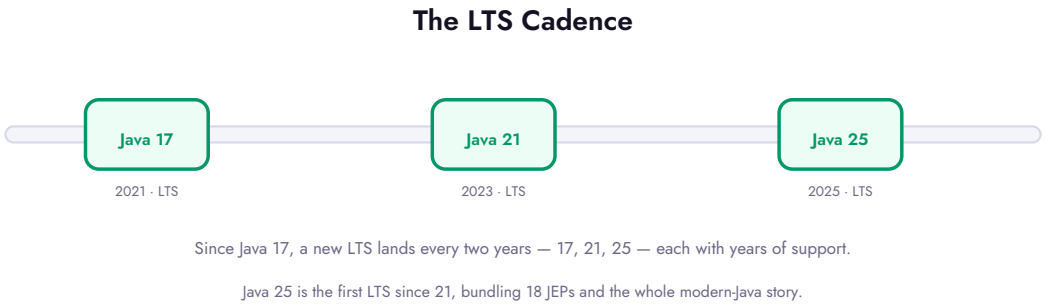
CHAPTER 01

Java 25 & the LTS Model

Why Java 25 matters as the newest LTS, and how the two-year LTS cadence works.

Java 25, released in September 2025, is the newest **Long-Term Support** (LTS) release and the first LTS since Java 21. For teams choosing a version to build on for years, it is the release to target — and it consolidates a huge amount of modern Java.

The LTS cadence



Since Java 17, a new LTS lands every two years — 17, 21, 25 — each supported for years.

Java still ships a feature release every six months, but since Java 17 an **LTS** is designated every two years: 17, 21, and now 25. LTS releases receive years of updates, which is why they are what most organisations run in production.

Why 25 specifically

- **The first LTS since 21** — it bundles everything finalized in 22, 23, 24, and 25.
- **18 JEPs** — spanning the language, libraries, and the runtime.
- **The modern-Java payoff** — virtual threads, pattern matching, and records are all mature here.
- **Years of support** — a stable base with an end-of-life around 2030.

Final vs preview

Java 25's 18 JEPs are a mix: some are **finalized** features you can use in production (scoped values, module imports, compact object headers), while others are **preview** or **incubator** features (structured concurrency, stable values) that need explicit flags and may still change. This guide always makes the status clear.

> **The newest LTS to target**
> Java 25 is the newest LTS, the first since 21, delivering 18 JEPs across language, libraries, and runtime. It consolidates the modern-Java story and gets years of support — the natural target for teams upgrading from 8, 11, 17, or 21.

© 2026 ByteMart Digital

MODULE 1 · THE JAVA 25 LANDSCAPE

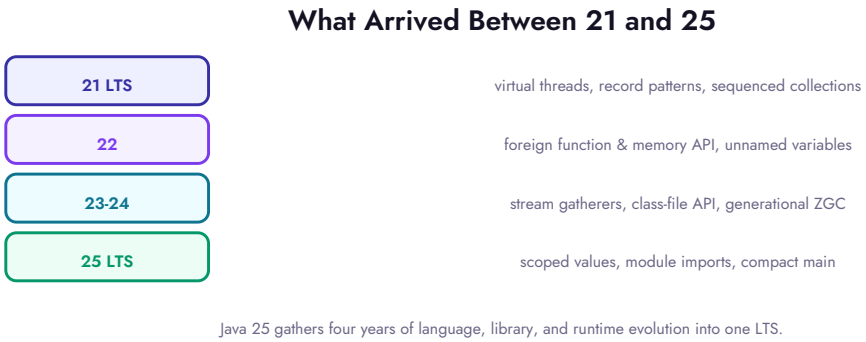
CHAPTER 02

The Journey from 21 to 25

A map of the major features finalized across the versions since the last LTS.

Because Java 25 is the first LTS since 21, upgrading to it means adopting everything that landed in between. This chapter maps the landmarks so you know what is new since the last LTS.

The journey



Java 25 gathers four years of language, library, and runtime evolution into one LTS.

Version	Landmark additions
21 LTS	virtual threads, record patterns, sequenced collections
22	foreign function & memory API, unnamed variables
23-24	stream gatherers, class-file API, generational ZGC
25 LTS	scoped values, module imports, compact main methods

Three themes

The additions cluster around three themes: **concurrency** (virtual threads, structured concurrency, scoped values from Project Loom), **language ergonomics** (pattern matching, records, simpler syntax), and **runtime performance** (compact object headers, better garbage collectors, faster startup).

How this guide is organised

The guide follows those themes: first the landscape and simpler syntax, then virtual threads and concurrency, then mature pattern matching, then modern language and library features, then performance and runtime, and finally migration. By the end you will write Java the way Java 25 intends.

> **Four years in one upgrade**
> Moving to Java 25 adopts everything since Java 21: virtual threads and the Loom concurrency model, matured pattern matching and records, and major runtime performance work. The guide walks these themes in order.

MODULE 1 · THE JAVA 25 LANDSCAPE

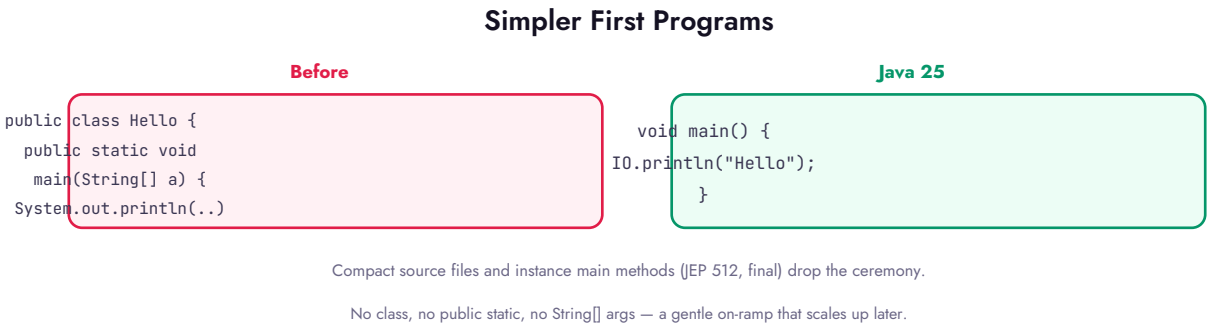
CHAPTER 03

Simpler First Programs

Compact source files and instance main methods that lower the barrier to entry.

Java's classic 'Hello World' forced beginners to meet `public`, `static`, `String[] args`, and class declarations on day one. **Compact source files and instance main methods** (JEP 512, final in Java 25) remove that ceremony.

The new form



Compact source files and instance main methods drop the ceremony — a gentle on-ramp.

```java
// the entire file:
void main() {
    IO.println("Hello, World!");
}
```

No class declaration, no `public static`, no `String[] args`. You can write a runnable program as just a `main` method, and the new `java.lang.IO` class offers `IO.println` and `IO.readln` for simple console work.

It scales up

This is not a separate dialect. As a program grows, you add fields, methods, and eventually a class declaration — the compact form quietly becomes ordinary Java. Beginners start simple and expand into the full language as they learn.

Who it helps

- **Beginners** — a first program without unexplained keywords.
- **Scripting and demos** — quick single-file programs.
- **Teaching** — focus on logic before boilerplate.

Hello World without ceremony

Compact source files and instance main methods (final in Java 25) let you write a program as just void main() with IO.println — no class, static, or args. It is a real on-ramp that expands seamlessly into full Java.

MODULE 1 · THE JAVA 25 LANDSCAPE

CHAPTER 04

Module Imports & Flexible Constructors

Two finalized language conveniences: importing whole modules and richer constructor bodies.

Java 25 finalizes two smaller but welcome language features that remove long-standing friction: importing everything a module exports in one line, and running statements before a constructor calls `super`.

Module import declarations

Module import declarations (JEP 511, final) let a single line import all the packages a module exports, instead of many individual imports. Your code does not even need to be modular to use it.

java

```
import module java.base;   // all packages exported by java.base

void main() {
    var list = new ArrayList<String>();   // no per-package imports
    IO.println(list);
}
```

Flexible constructor bodies

Flexible constructor bodies (JEP 513, final) allow statements before the call to `super(...)` or `this(...)` — so you can validate or prepare arguments first, which the old 'super must come first' rule forbade.

java

```
class PositiveInt extends Number {
    PositiveInt(int value) {
        if (value < 0)                    // validate BEFORE super()
            throw new IllegalArgumentException("must be >= 0");
        super();
    }
}
```

Why they matter

Both changes remove awkward workarounds — static helper methods to validate constructor arguments, or long lists of imports. Small conveniences, but they make everyday code cleaner and are safe to adopt immediately since they are final.

Two clean, final conveniences

Module import declarations (import module java.base) pull in all of a module's exported packages in one line, and flexible constructor bodies allow validation before super() or this(). Both are finalized in Java 25 and safe to use now.

© 2026 ByteMart Digital

MODULE 2 · VIRTUAL THREADS & CONCURRENCY

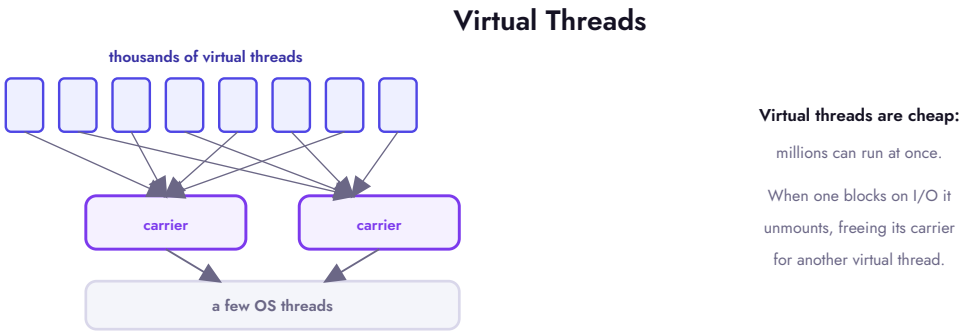
CHAPTER 05

Virtual Threads: The Big Idea

The Project Loom feature that makes highly concurrent code simple and cheap.

Virtual threads (final in Java 21, part of Project Loom) are the most important concurrency change in Java's history. They make it possible to run millions of concurrent tasks with simple, blocking code — and they are mature in Java 25.

Many threads, few OS threads



Write simple blocking code; the runtime multiplexes many virtual threads onto few OS threads.

The runtime multiplexes many cheap virtual threads onto a few OS threads.

A traditional **platform thread** maps one-to-one to an expensive OS thread, so you can only have a few thousand. A **virtual thread** is lightweight and managed by the JVM — you can have millions, and the runtime multiplexes them onto a small pool of OS carrier threads.

Blocking becomes cheap

The key trick: when a virtual thread blocks on I/O, it **unmounts** from its carrier, freeing that OS thread to run another virtual thread. So the old cost of blocking — a tied-up OS thread — disappears. You write simple, blocking, sequential code and get massive scalability.

Why this changes everything

- **Simple code, high scale** — no callbacks or reactive complexity needed.
- **A thread per request** — the natural model becomes viable again.
- **Cheap to create** — spawn one per task without a pool.

Millions of cheap threads

Virtual threads are lightweight, JVM-managed threads multiplexed onto a few OS threads. When one blocks on I/O it unmounts, so blocking is cheap. You write simple thread-per-task code and scale to millions of concurrent operations.

MODULE 2 · VIRTUAL THREADS & CONCURRENCY

CHAPTER 06

Using Virtual Threads

The small API for creating virtual threads and running tasks on them.

The virtual-thread API is deliberately tiny — they are still `Thread` objects, so most existing code works unchanged. Here is how to create them and run tasks.

Creating a virtual thread

java

```
// start one directly
Thread.startVirtualThread(() -> {
    IO.println("on a virtual thread");
});

// or build one
Thread t = Thread.ofVirtual().name("worker").start(task);
```

An executor per task

The idiomatic approach is an executor that creates a **new virtual thread for every task** — no pool, no reuse, because virtual threads are cheap to create and discard.

java

```
try (var executor =
        Executors.newVirtualThreadPerTaskExecutor()) {
    for (var task : tasks) {
        executor.submit(() -> handle(task));   // one vthread each
    }
}   // try-with-resources waits for all to finish
```

Everything else is the same

Virtual threads support the same operations as platform threads — `sleep`, synchronisation, thread locals, blocking I/O. Existing libraries and frameworks that spawn tasks can run on them with little or no change.

TIP

Most modern frameworks can run request handlers on virtual threads with a single configuration switch — often the only change needed to benefit from them.

A tiny, familiar API

Create virtual threads with Thread.startVirtualThread or Thread.ofVirtual, or use Executors.newVirtualThreadPerTaskExecutor for one thread per task. They are ordinary Thread objects, so existing blocking code and libraries work unchanged.

MODULE 2 · VIRTUAL THREADS & CONCURRENCY

CHAPTER 07

Virtual Threads in Practice

The pitfalls to avoid and the situations where virtual threads shine.

Virtual threads are powerful, but a few habits from the platform-thread era actively hurt them. Knowing the pitfalls is the difference between a smooth adoption and a puzzling one.

Do not pool them

WATCH OUT

Never put virtual threads in a fixed pool. Pooling exists to reuse expensive platform threads; virtual threads are cheap, so pooling them defeats the purpose. Create a new one per task — that is the whole point.

Watch for pinning

A virtual thread cannot unmount while inside a `synchronized` block or a native call — it is **pinned** to its carrier, blocking it. Recent releases reduce this, but for hot paths prefer `ReentrantLock` over `synchronized` to avoid pinning.

Where they shine

Great for	Not the point
I/O-bound tasks	CPU-bound number crunching
many concurrent requests	a few long computations
blocking calls (DB, HTTP)	raw parallel throughput

The mental model

Use virtual threads for **concurrency** — waiting on many things at once, like network and database calls. For **parallelism** — maximising CPU on heavy computation — platform threads and parallel streams still fit. Virtual threads win where code spends its time waiting.

Create freely, avoid pinning

Do not pool virtual threads — create one per task. Avoid pinning by preferring ReentrantLock over synchronized on hot paths. They excel at I/O-bound concurrency with many waiting tasks, not CPU-bound parallelism.

© 2026 ByteMart Digital

MODULE 2 · VIRTUAL THREADS & CONCURRENCY

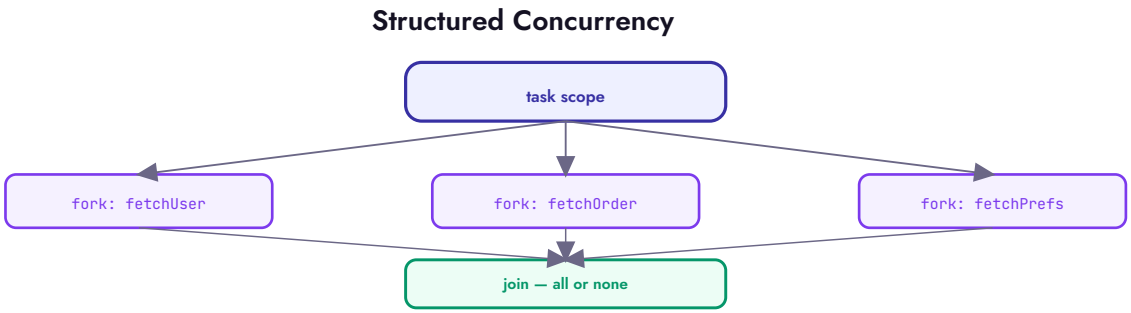
CHAPTER 08

Structured Concurrency

Treating a group of concurrent subtasks as a single, reliable unit of work.

When one task spawns several concurrent subtasks, managing their lifetimes, errors, and cancellation by hand is error-prone. **Structured concurrency** (JEP 505, a preview in Java 25) treats them as one unit — they start, join, and cancel together.

A scope for related tasks



Related subtasks form one unit: they start together, join together, and cancel together (preview).

Related subtasks form one unit: they start together, join together, and cancel together.

```java
// preview API — shape may still change
try (var scope = StructuredTaskScope.open()) {
    var user  = scope.fork(() -> fetchUser(id));
    var order = scope.fork(() -> fetchOrder(id));
    scope.join();                       // wait for both
    return new Response(user.get(), order.get());
}   // leaving the scope cancels anything still running
```

Why it is safer

- **No leaks** — subtasks cannot outlive their scope.
- **All-or-none** — if one fails, the rest are cancelled.
- **Clear structure** — concurrency follows the code's block structure.

Pairs with virtual threads

Structured concurrency forks each subtask on a virtual thread, so forking many is cheap. Together they make concurrent code read like ordinary sequential code, with reliable error handling built in.

WATCH OUT

Structured concurrency is a **preview** feature in Java 25 (its fifth), so it needs `--enable-preview` and its exact API may still change. Learn the model now; pin to the final API when it lands.

Concurrency with structure

Structured concurrency groups related subtasks in a scope so they start, join, and cancel as one unit — no leaks, all-or-none failure. It pairs naturally with virtual threads, but remains a preview in Java 25.

© 2026 ByteMart Digital

MODULE 2 · VIRTUAL THREADS & CONCURRENCY

CHAPTER 09

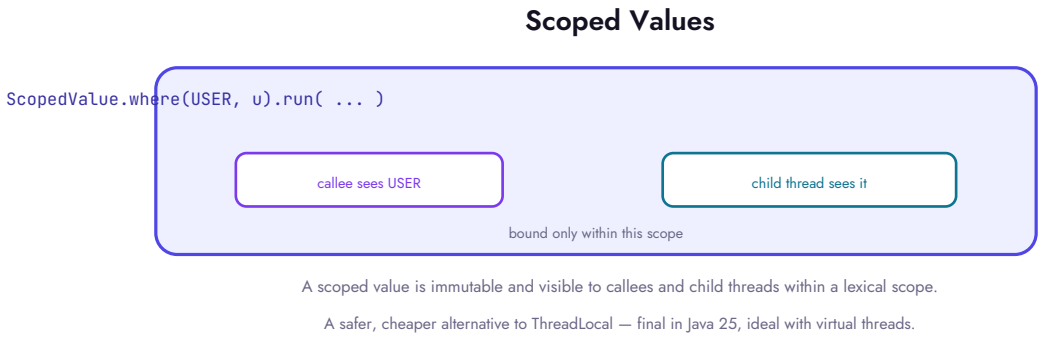
Scoped Values

A safer, cheaper way to share immutable context than ThreadLocal.

09

Sharing context — a current user, a request ID — across method calls often meant `ThreadLocal`, which is mutable, leak-prone, and awkward with virtual threads. **Scoped values** (JEP 506, **final** in Java 25) are the modern replacement.

A value bound to a scope



A scoped value is immutable and visible to callees and child threads within a lexical scope.

```java
final static ScopedValue<User> CURRENT_USER =
    ScopedValue.newInstance();

ScopedValue.where(CURRENT_USER, user).run(() -> {
    handleRequest();          // any callee can read it
});

// deep inside the call chain:
User u = CURRENT_USER.get();
```

Why it beats ThreadLocal

ThreadLocal	ScopedValue
mutable, set anytime	immutable, bound once
unclear lifetime, can leak	bounded to a scope
costly with virtual threads	cheap, ideal with them

Bounded and safe

A scoped value is set only for the duration of the `run` block and is automatically unbound when it exits — no cleanup, no stale data, no leaks. It is visible to every method called within, and to child threads forked in that scope.

Immutable context, done right

Scoped values (final in Java 25) share immutable context within a lexical scope, visible to callees and child threads and unbound automatically on exit. They replace ThreadLocal — safer, leak-free, and cheap with virtual threads.

© 2026 ByteMart Digital

MODULE 3 · PATTERN MATCHING MATURED

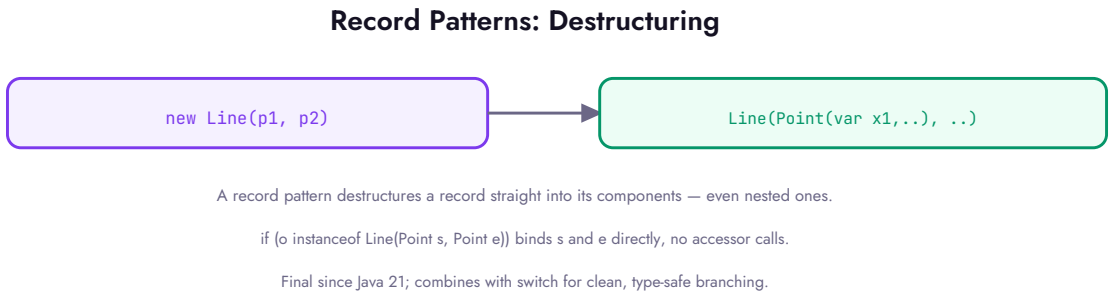
CHAPTER 10

Record Patterns

Destructuring records directly in instanceof and switch, even when nested.

Records model data concisely; **record patterns** (final in Java 21) let you take them apart just as concisely. Instead of calling accessors, you destructure a record straight into its components — even nested ones.

Destructuring



A record pattern destructures a record into its components — even nested ones.

```java
record Point(int x, int y) {}
record Line(Point start, Point end) {}

if (obj instanceof Line(Point(var x1, var y1),
                        Point(var x2, var y2))) {
    double len = Math.hypot(x2 - x1, y2 - y1);
}
```

The pattern `Line(Point start, Point end)` matches a `Line` and binds its components in one step. Nesting the patterns reaches inside `Point` too — no chain of `getStart().getX()` calls.

With switch

```java
String describe(Object o) {
    return switch (o) {
        case Point(var x, var y) -> "point " + x + "," + y;
        case Line(var s, var e)  -> "line";
        default                  -> "unknown";
    };
}
```

Why it matters

Record patterns turn data inspection into clear, declarative code. Combined with sealed types and switch, they let the compiler check that you have handled every case — the foundation of the algebraic-data-type style covered shortly.

Take records apart declaratively

Record patterns (final since Java 21) destructure a record straight into its components in instanceof and switch, including nested records — no accessor chains. They set up exhaustive, compiler-checked matching over sealed types.

© 2026 ByteMart Digital

MODULE 3 · PATTERN MATCHING MATURED

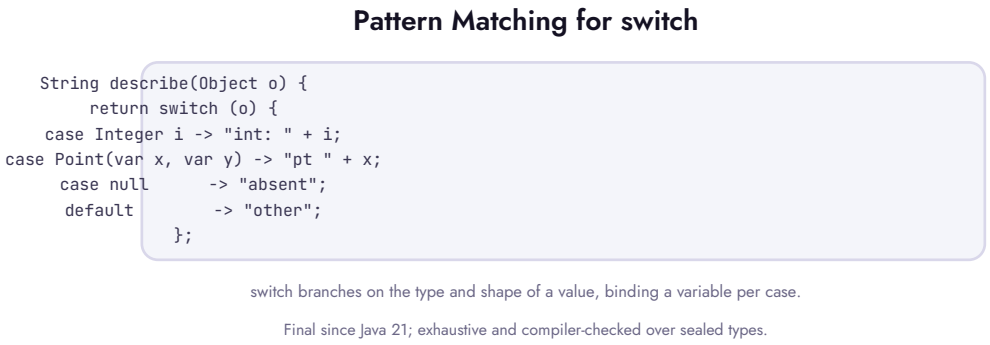
CHAPTER 11

Pattern Matching for switch

Branching on the type and shape of a value, exhaustively checked by the compiler.

Pattern matching for switch (final in Java 21) lets a `switch` branch on a value's type and shape, binding a variable per case. It replaces long chains of `if-else instanceof` with clear, exhaustive code.

Switching on type



switch branches on the type and shape of a value, binding a variable per case.

```java
double area(Shape s) {
    return switch (s) {
        case Circle c   -> Math.PI * c.radius() * c.radius();
        case Square sq  -> sq.side() * sq.side();
        case Line l     -> 0;
    };
}
```

Guards

A `when` guard adds a condition to a case, so you can distinguish values of the same type — for example separating positive from negative numbers within the switch.

```java
case Integer i when i > 0 -> "positive";
case Integer i            -> "non-positive";
```

Exhaustiveness over sealed types

When you switch over a **sealed** type, the compiler knows every permitted subtype. Cover them all and no `default` is needed; miss one and it is a compile error. This safety is the reward for sealing your hierarchies.

Branch on type, exhaustively

Pattern matching for switch (final since Java 21) branches on a value's type and shape, with when guards for extra conditions. Over a sealed type it is exhaustive and compiler-checked — the modern replacement for if-instanceof chains.

MODULE 3 · PATTERN MATCHING MATURED

CHAPTER 12

Primitive Types in Patterns

Extending pattern matching to primitive types — a Java 25 preview.

Pattern matching originally worked only with reference types. **Primitive types in patterns** (JEP 507, a preview in Java 25) extends `instanceof` and `switch` to work with primitives like `int` and `long` too.

instanceof with primitives

```java
// preview feature
static void test(Object obj) {
    if (obj instanceof int i) {
        IO.println("an int: " + i);
    }
}
```

switch with primitive patterns

It also lets a `switch` match and bind primitive values directly, with guards, avoiding manual unboxing and range checks.

```java
String classify(int n) {
    return switch (n) {
        case 0            -> "zero";
        case int i when i > 0 -> "positive";
        default           -> "negative";
    };
}
```

Why it helps

- **Uniformity** — the same pattern syntax for primitives and objects.
- **Less boilerplate** — no manual boxing, casting, or range checks.
- **Type-safe** — the compiler verifies the conversions.

WATCH OUT

This is a **preview** feature in Java 25 (its third), enabled with `--enable-preview`. The design is stable but not yet permanent — use it to explore; expect it to finalize in a later release.

© 2026 ByteMart Digital

Patterns for primitives too

Primitive types in patterns (a preview in Java 25) extends instanceof and switch to match and bind primitives like int and long, with guards — uniform syntax and no manual unboxing. It remains a preview, so enable it explicitly.

© 2026 ByteMart Digital

MODULE 3 · PATTERN MATCHING MATURED

CHAPTER 13

Sealed + Records + Patterns

How three mature features combine into algebraic data types with compiler-checked completeness.

Records, sealed types, and pattern matching are strong alone, but together they bring **algebraic data types** to Java: a closed set of variants, each a concise record, handled by an exhaustive switch the compiler verifies. In Java 25, all three are mature and final.

The combination

java

```
sealed interface Payment permits Card, Upi, Cash {}
record Card(String number) implements Payment {}
record Upi(String vpa)     implements Payment {}
record Cash()              implements Payment {}

String describe(Payment p) {
    return switch (p) {              // exhaustive: no default
        case Card(var n) -> "card ending " + n;
        case Upi(var v)  -> "upi " + v;
        case Cash()      -> "cash";
    };
}
```

Why it is powerful

- **Sealed** fixes the complete set of variants.
- **Records** model each variant concisely and immutably.
- **Record patterns** destructure each one in the switch.
- **The compiler** errors if you add a variant and forget to handle it.

Compiler-enforced completeness

Add a new `Payment` subtype and every exhaustive switch over `Payment` that does not handle it fails to compile. The compiler points you at every place to update — a safety net hand-written type checks never gave you.

> **Algebraic data types in Java**
> Sealed interfaces of records, matched by an exhaustive switch with record patterns, give Java algebraic data types: a closed, immutable variant set with compiler-enforced completeness. All three features are mature and final in Java 25.

MODULE 4 · MODERN LANGUAGE & LIBRARY

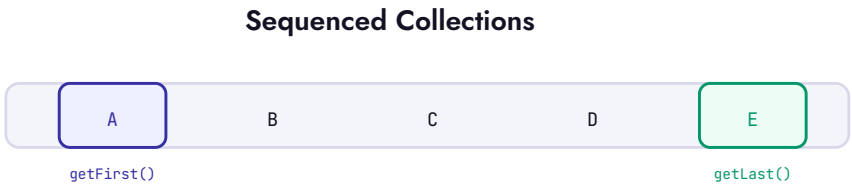
CHAPTER 14

Sequenced Collections

A unified way to work with the first and last elements of any ordered collection.

For years, getting the first or last element of an ordered collection was inconsistent — `list.get(0)` here, an iterator trick there. **Sequenced collections** (final in Java 21) add a uniform interface for this.

First, last, reversed



A unified interface gives every ordered collection getFirst(), getLast(), and reversed().

```java
SequencedCollection<String> list =
    new ArrayList<>(List.of("a", "b", "c"));

list.getFirst();     // "a"
list.getLast();      // "c"
list.addFirst("z");  // prepend
list.reversed();     // a reverse-ordered view
```

The new interfaces

Interface	Adds
SequencedCollection	getFirst/getLast, addFirst/addLast, reversed
SequencedSet	the above for ordered sets
SequencedMap	firstEntry/lastEntry, reversed view

Why it helps

`List`, `Deque`, `LinkedHashSet`, and `LinkedHashMap` all gain these methods through the new interfaces — one consistent, readable vocabulary for the ends of any ordered collection, replacing a grab-bag of older idioms.

© 2026 ByteMart Digital

Uniform access to the ends

Sequenced collections (final since Java 21) give ordered collections a consistent interface — getFirst, getLast, addFirst, addLast, and reversed — across List, Deque, and the Linked sets and maps, replacing inconsistent older idioms.

© 2026 ByteMart Digital

MODULE 4 · MODERN LANGUAGE & LIBRARY

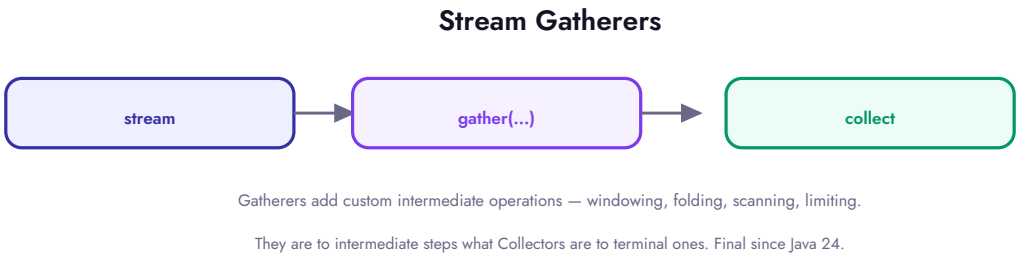
CHAPTER 15

Stream Gatherers

Building custom intermediate stream operations the way Collectors build terminal ones.

The Streams API had a fixed set of intermediate operations — `map`, `filter`, and a few more. **Stream gatherers** (final in Java 24) let you write your own intermediate operations, the way `Collectors` let you write terminal ones.

The gather operation



Gatherers add custom intermediate operations — windowing, folding, scanning, limiting.

```java
// group a stream into fixed-size windows
List<List<Integer>> windows = Stream.of(1,2,3,4,5,6,7)
    .gather(Gatherers.windowFixed(3))
    .toList();
// [[1,2,3], [4,5,6], [7]]
```

Built-in gatherers

Gatherer	Does
windowFixed(n)	group into non-overlapping windows
windowSliding(n)	group into sliding windows
fold / scan	accumulate, emitting results
mapConcurrent	map with bounded concurrency

Why it matters

Before gatherers, operations like windowing or running totals meant clumsy workarounds or leaving the stream entirely. Gatherers make these first-class and composable — and you can write custom ones for domain-specific processing. They complete the Streams API's extensibility.

Custom intermediate operations

Stream gatherers (final in Java 24) let you add custom intermediate operations via gather(), with built-ins for windowing, folding, scanning, and bounded-concurrency mapping. They are to intermediate steps what Collectors are to terminal ones.

MODULE 4 · MODERN LANGUAGE & LIBRARY

CHAPTER 16

The Modern Java Basics

A fast recap of the core modern features you inherit when you reach Java 25.

If you are arriving from Java 8 or 11, Java 25 hands you a decade of language evolution at once. This chapter is a quick recap of the everyday modern features you should adopt — covered in depth in dedicated guides, summarised here.

Records and sealed types

Records declare immutable data carriers in one line, generating the constructor, accessors, and `equals/hashCode/toString`. **Sealed** types restrict which classes may extend them, enabling the exhaustive pattern matching seen earlier.

```java
record User(Long id, String name) {}
sealed interface Shape permits Circle, Square {}
```

Text blocks and switch expressions

Text blocks give clean multi-line strings for JSON, SQL, and HTML. **Switch expressions** return a value with arrow labels and no fall-through. Both are everyday tools in modern Java.

```java
String json = """
    { "id": 1, "name": "Ada" }
    """;
int n = switch (day) { case MON -> 1; default -> 0; };
```

var and the rest

- **var** — local variable type inference where the type is obvious.
- **Enhanced instanceof** — test and bind in one step.
- **New String and Stream methods** — `strip`, `toList()`, and more.

> **A decade of features, inherited**
> Reaching Java 25 from 8 or 11 hands you records, sealed types, text blocks, switch expressions, var, and enhanced instanceof — the everyday modern basics. Adopt them alongside the newer Java 25 features.

© 2026 ByteMart Digital

MODULE 4 · MODERN LANGUAGE & LIBRARY

CHAPTER 17

Stable Values

A preview API for lazy, set-once constants the JVM can optimise like final fields.

Some values are expensive to build and best created lazily — but `final` fields must be set at construction, and lazy-initialisation code is verbose and error-prone. **Stable values** (JEP 502, a preview in Java 25) give a clean middle ground.

A set-once holder

```java
// preview feature
class OrderController {
    private final StableValue<Logger> logger = StableValue.of();

    Logger getLogger() {
        return logger.orElseSet(
            () -> Logger.create(OrderController.class));
    }
}
```

A `StableValue` is empty until first set, then immutable. `orElseSet` computes and stores the value on first access and returns the cached one thereafter — lazy initialisation without the usual boilerplate or race conditions.

Treated as a constant

Crucially, once set, the JVM can treat a stable value as a **constant** — unlocking the same optimisations as a `final` field, which ordinary lazy holders cannot get. You gain laziness without losing performance.

final vs StableValue

final field	StableValue
set at construction	set lazily, on first use
eager	deferred until needed
constant to the JVM	also constant once set

WATCH OUT

Stable values are a **preview** in Java 25, enabled with `--enable-preview`. Explore the API, but do not depend on its exact shape in production until it is finalized.

Lazy constants, JVM-friendly

Stable values (a preview in Java 25) hold set-once data initialised lazily via orElseSet, without lazy-init boilerplate or races — and once set, the JVM treats them as constants like final fields. Enable the preview to use them.

MODULE 4 · MODERN LANGUAGE & LIBRARY

CHAPTER 18

Foreign Function & Memory API

Calling native code and accessing off-heap memory safely, without JNI.

Calling native libraries from Java once meant **JNI** — verbose, fragile, and unsafe. The **Foreign Function and Memory API** (JEP 454, final in Java 22) replaces it with a safe, pure-Java way to call native code and manage off-heap memory.

Two capabilities

- **Foreign functions** — call C-style native library functions directly from Java.
- **Foreign memory** — allocate and access off-heap memory safely, with deterministic cleanup.

Managing native memory

An `Arena` owns a region of off-heap memory and frees it deterministically when closed — no manual `free`, no leaks. `MemorySegment` gives bounds-checked access to that memory, so the classic native-code crashes are caught as Java exceptions.

java

```
try (Arena arena = Arena.ofConfined()) {
    MemorySegment seg = arena.allocate(1024);
    seg.set(ValueLayout.JAVA_INT, 0, 42);
    int v = seg.get(ValueLayout.JAVA_INT, 0);
}   // memory freed automatically here
```

Why it matters

The FFM API makes integrating with native libraries — for performance, hardware, or existing C code — far safer and simpler than JNI. It is the modern foundation for native interop and off-heap data, final and production-ready since Java 22.

Safe native interop, no JNI

The Foreign Function and Memory API (final since Java 22) calls native functions and manages off-heap memory in pure Java. Arena frees memory deterministically and MemorySegment is bounds-checked — replacing JNI with a safe, modern alternative.

© 2026 ByteMart Digital

MODULE 5 · PERFORMANCE & RUNTIME

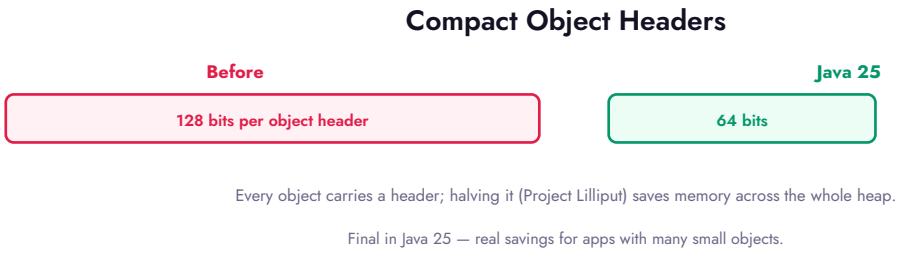
CHAPTER 19

Compact Object Headers

Halving the per-object header to cut memory use across the whole heap.

Every Java object carries a hidden **header** of metadata. **Compact object headers** (JEP 519, final in Java 25) shrink it from 128 bits to 64 bits on 64-bit JVMs — a small change per object that adds up across millions.

Smaller headers, less memory



Halving the object header (Project Lilliput) saves memory across the whole heap.

The header stores things like the identity hash and locking state. Historically it took 96-128 bits; compact headers pack this into 64 bits. For applications with **many small objects**, the cumulative saving in heap memory — and the resulting CPU and cache benefits — is significant.

From experimental to final

Compact object headers arrived as an experimental feature in Java 24 and are **final** in Java 25, having proven stable. They come from **Project Lilliput**, whose goal is to substantially reduce Java's memory footprint.

Enabling it

```text
java -XX:+UseCompactObjectHeaders -jar app.jar
```

It is opt-in via a flag. For memory-intensive services — large caches, many entities, big object graphs — it is worth benchmarking, as the reduced footprint can lower memory cost and improve cache behaviour.

Half-size headers, real savings

Compact object headers (final in Java 25, from Project Lilliput) shrink the per-object header from 128 to 64 bits on 64-bit JVMs. Enabled with a flag, they cut heap memory for apps with many small objects, improving CPU and cache performance.

MODULE 5 · PERFORMANCE & RUNTIME

CHAPTER 20

Modern Garbage Collectors

Choosing among G1, ZGC, and Shenandoah — all now generational and low-pause.

Java's garbage collectors have improved dramatically. By Java 25 you have several strong, low-pause options, and the newest ones are **generational** — optimising short-lived and long-lived objects separately for better throughput and shorter pauses.

The main collectors

Collector	Best for
G1 (default)	balanced throughput and pauses
ZGC	very large heaps, ultra-low pauses
Shenandoah	low pauses, now generational
Parallel	raw throughput, batch jobs

Generational everywhere

Most objects die young, so collecting a **young generation** separately is far more efficient. Generational ZGC (final in Java 24) and **generational Shenandoah** (JEP 521, final in Java 25) bring this efficiency to the low-pause collectors, matching G1's generational design while keeping tiny pauses.

Choosing one

- **Start with G1** — the default, well-balanced for most apps.
- **Use ZGC** for huge heaps where pause time is critical.
- **Use Shenandoah** for consistently low pauses; enable generational mode.
- **Measure** — pick based on your latency and throughput goals.

```text
java -XX:+UseZGC -jar app.jar          # ultra-low pause
java -XX:+UseShenandoahGC \
     -XX:ShenandoahGCMode=generational -jar app.jar
```

Strong, generational collectors

Java 25 offers G1 (default), ZGC, and Shenandoah, all low-pause. Generational ZGC (24) and generational Shenandoah (25) collect young objects separately for better efficiency. Start with G1; choose ZGC or Shenandoah for large heaps or strict pauses, and measure.

© 2026 ByteMart Digital

MODULE 5 · PERFORMANCE & RUNTIME

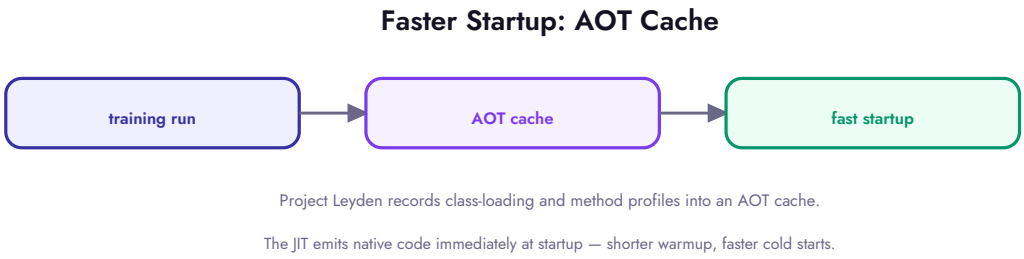
CHAPTER 21

Faster Startup: AOT & Leyden

Cutting warmup and cold-start time with ahead-of-time caching and profiling.

Java is fast once warmed up, but startup and warmup have long been weak points — painful for microservices and serverless functions. **Project Leyden**, landing in Java 25, tackles this with ahead-of-time (AOT) caching and profiling.

The AOT cache



Leyden records class-loading and method profiles into an AOT cache for faster startup.

A **training run** records how your application loads classes and executes methods, storing this in an **AOT cache**. Later runs reuse the cache to skip repeated work and start faster. JEP 514 simplifies creating the cache to a single command.

```text
# create the cache from a training run
java -XX:AOTCacheOutput=app.aot -jar app.jar

# use it for fast startup
java -XX:AOTCache=app.aot -jar app.jar
```

Ahead-of-time method profiling

AOT method profiling (JEP 515) stores method-execution profiles from an earlier run in the cache. At startup the JIT can immediately generate optimised native code, instead of waiting to gather profiles — dramatically shortening warmup.

Why it matters

For short-lived cloud workloads — serverless functions, autoscaling microservices — startup time directly affects cost and responsiveness. Leyden's faster cold starts and warmup make Java far more competitive for these deployments.

Shorter startup and warmup

Project Leyden (Java 25) records class-loading and method profiles into an AOT cache, created with a single command (JEP 514) and reused for faster startup. AOT method profiling (JEP 515) lets the JIT emit native code immediately — key for serverless and microservices.

MODULE 5 · PERFORMANCE & RUNTIME

CHAPTER 22

Observability: JFR & Class-File API

Better built-in profiling with JDK Flight Recorder, and a standard way to process bytecode.

Java 25 also strengthens the tools around your code: **JDK Flight Recorder** gains new profiling abilities, and the **Class-File API** gives a standard, supported way to read and write bytecode.

JDK Flight Recorder improvements

JFR is Java's low-overhead, built-in profiler. Java 25 adds **method timing and tracing** (JEP 520) to measure and trace specific methods, more accurate **CPU-time profiling** on Linux (JEP 509, experimental), and **cooperative sampling** (JEP 518) for more stable stack sampling.

- **Method timing & tracing** — target specific methods for measurement.
- **CPU-time profiling** — more accurate hotspots on Linux (experimental).
- **Cooperative sampling** — steadier, safer thread-stack sampling.

The Class-File API

Tools and frameworks that read or generate bytecode long relied on third-party libraries that lagged behind each Java release. The **Class-File API** (JEP 484, final in Java 24) is a standard JDK API for parsing, generating, and transforming class files — always current with the language.

```java
// standard, supported bytecode processing
ClassFile.of().parse(bytes)
    .methods()
    .forEach(m -> IO.println(m.methodName()));
```

Why it matters

Better observability means finding performance problems faster in production, and a standard Class-File API means the frameworks you depend on can support new Java versions sooner. Both make the ecosystem healthier.

Sharper tools around your code

Java 25 enhances JFR with method timing and tracing, better CPU-time profiling, and cooperative sampling for low-overhead production profiling. The standard Class-File API (final in Java 24) gives supported, always-current bytecode processing.

MODULE 6 · MIGRATION & PRACTICE

CHAPTER 23

Migrating to Java 25

The practical steps and gotchas when upgrading from an older release.

Upgrading to Java 25 is usually smoother than teams expect — Java prizes backward compatibility — but the path depends on where you start, and a few removals and flags catch people out.

Know your starting point

From	Main concerns
Java 8/11	removed EE modules, strong encapsulation
Java 17	mostly smooth; adopt new features
Java 21	very smooth; a few removals below

Java 25 removals to note

- **The 32-bit x86 port is removed** (JEP 503) — use a 64-bit JVM.
- **The experimental Graal JIT compiler is removed** — the default JIT remains.
- **Older deprecated APIs** — check for removals since your version.

Preview and incubator flags

To use Java 25's **preview** features (structured concurrency, stable values, primitive patterns) you need `--enable-preview` with `--release 25`. Incubator modules like the Vector API need `--add-modules`. Production code should generally stick to finalized features.

A migration order

Update your build and dependencies first — current library versions clear most issues — then compile on 25, run your tests, and review JVM flags. Get it **running** on 25 before adopting new features; modernise the code in a separate, calmer pass.

> **Migrate first, modernise later**
> Upgrading to Java 25 is mostly compatible; note the removed 32-bit x86 port and Graal JIT, and use –enable-preview only for preview features. Update dependencies, compile, test, and review flags to get running — then adopt new features separately.

© 2026 ByteMart Digital

MODULE 6 · MIGRATION & PRACTICE

CHAPTER 24

Best Practices & Modern Java Style

Habits for writing idiomatic Java that makes the most of the Java 25 LTS.

Running on Java 25 is one thing; writing modern Java is another. This closing chapter gathers the habits that make the most of everything the LTS offers.

Adopt the modern idioms

- **Virtual threads** for I/O-bound concurrency — one per task, no pools.
- **Scoped values** instead of ThreadLocal for request context.
- **Records and sealed types** with pattern-matching switch for data and logic.
- **Sequenced collections and stream gatherers** for cleaner data handling.
- **Switch expressions, var, and text blocks** as everyday defaults.

Prefer immutability

Modern Java leans immutable — records, scoped values, unmodifiable collections. Immutable data is easier to reason about and safe to share across the many virtual threads you will now run. Reach for it by default.

Be deliberate about preview features

WATCH OUT

Preview features (structured concurrency, stable values, primitive patterns) are worth learning, but keep them out of production until they finalize — their APIs can still change. Build production code on the finalized feature set.

Modernise gradually

You do not have to rewrite everything at once. As you touch old code, nudge it toward the modern idioms — a record here, virtual threads there, a pattern switch replacing an if-chain. Over time the codebase becomes cleaner without a risky big-bang rewrite.

Write Java the modern way

On Java 25, prefer virtual threads for concurrency, scoped values over ThreadLocal, records and sealed types with pattern switch, and sequenced collections and gatherers. Lean immutable, keep preview features out of production, and modernise gradually.

© 2026 ByteMart Digital